МЕТОДИЧЕСКИЕ УКАЗАНИЯ
к лабораторной работе № 3

Тема: «Применение управляющих конструкций для валидации данных: if-elif-else, match-case, циклический ввод»

Вариант А — базовая траектория · Вариант Б — продвинутая траектория
Работа рассчитана на 2 пары (4 академических часа)

Дисциплина: «Программирование и прототипирование ИТ-решений»

Направление подготовки: 09.03.03 Прикладная информатика

Профиль: Бизнес и системная аналитика

Семестр / трудоёмкость: 1 семестр, 4 академических часа (2 пары)

Среда разработки: Visual Studio Code

Форма контроля: отчёт о выполнении работы + устная защита

Обеспеченность ПО: Python 3.11+, Git, VS Code (подготовлены в ЛР № 1)

1. Цель работы

Научиться строить программу, которая принимает данные от пользователя, проверяет их корректность с помощью условных операторов, организует повторный ввод до получения допустимого значения и маршрутизирует логику через `match-case` — на примере валидации технологических параметров нефтяной скважины.

В результате выполнения работы студент научится:

- организовывать циклический ввод (`while True` + `break`) до получения корректного значения, не падая на ошибках пользователя;
- проверять границы допустимых значений через `if-elif-else` и сообщать пользователю, ЧТО именно не так (меньше минимума / больше максимума / не число);
- классифицировать объекты по диапазонам значений цепочкой `if-elif-else`;
- маршрутизировать логику программы конструкцией `match-case` (меню, статусы);
- выбирать уместную конструкцию: когда `if-elif-else`, а когда `match-case`;
- документировать проверку: ручной прогноз до запуска, скриншоты провокационных вводов, таблица «прогноз | факт».

2. Обязательное ядро (одинаково для обоих вариантов)

- скрипт запускается из командной строки и взаимодействует с пользователем через `input()`;
- числовой ввод преобразуется через `float()` (при необходимости `int()`), запятая `3,14` приводится к точке;
- каждое значение проверяется ДО использования: `if-elif-else` проверяет границы и выдаёт конкретное сообщение об ошибке;

- ввод повторяется циклически (`while True` + `break`/`continue`) до получения допустимого значения;
- логика маршрутизируется конструкцией `match-case` (меню и/или статусы);
- программа вычисляет и классифицирует результат, а не печатает заготовку;
- в репозитории Git сделано не менее трёх осмысленных коммитов (инициализация → рабочий код → документация);
- в отчёте есть ручной прогноз (предсказание ДО запуска) и таблица «прогноз | факт» для ВАШИХ индивидуальных данных;
- выполнены ВСЕ обязательные упражнения на понимание (раздел 9) с фиксацией проверки понимания в отчёте;
- README.md содержит назначение, пример запуска и пример ввода/вывода.

3. Глоссарий: все термины работы

Перед началом работы прочитайте термины. К ним можно возвращаться по ходу работы.

3.1. Условные операторы и границы значений

Термин	Что это значит
`if-elif-else`	ветвление: программа выполняет ОДНУ из нескольких веток по истинности условий. `elif` — «иначе если», `else` — «во всех остальных случаях».
Условие	выражение, дающее `True`/`False`: `x < 0`, `w >= 5`.
Операторы сравнения	`>`, `<`, `>=`, `<=`, `==` (равно), `!=` (не равно).
Логические операторы	`and` (и), `or` (или), `not` (не) — комбинируют условия.
Цепочка сравнения	`0 <= x <= 100` — то же, что `0 <= x and x <= 100`.
Валидация	проверка данных на корректность (тип, диапазон) до их использования.
Границы / диапазон	допустимый интервал значений, например дебит — от 0 до 1000 м³/сут.

3.2. Конструкция match-case

Термин	Что это значит
`match` / `case`	структурное сопоставление (Python 3.10+): выбирает ветку по ЗНАЧЕНИЮ выражения, а не по условию.
Ветка `case`	шаблон, с которым сравнивается значение: `case "1":`.
`case _`	ветка «по умолчанию» — срабатывает, если ни один шаблон не подошёл.
Маршрутизация	направление выполнения программы по одному из готовых сценариев (пункт меню, статус).
Охранник `case ... if`	дополнительное условие к шаблону: `case x if x < 0:`.

3.3. Циклический ввод и устойчивость

Термин	Что это значит
`while True:`	бесконечный цикл: повторяется, пока не будет явного `break`.
`break`	немедленно выходит из цикла — «значение принято, хватит

	спрашивать».
`continue`	переходит к следующей итерации цикла, не выполняя остаток тела.
`try/except`	перехватывает ошибки: при `ValueError` в `try` выполняется `except`.
`replace(",", ".")`	заменяет запятую точкой, чтобы `0,25` читалось как число.
`strip()`	убирает пробелы и перевод строки по краям строки ввода.

4. Необходимое программное обеспечение

Всё окружение подготовлено в лабораторной работе № 1. Убедитесь, что оно на месте:

Инструмент	Назначение	Проверка
Python 3.10+	интерпретатор (`match-case` требует 3.10+)	`python --version`
Visual Studio Code	редактор и терминал	окно VS Code открывается
Git	система контроля версий	`git --version`
Виртуальное окружение	изоляция зависимостей проекта	`(.venv)` в начале строки

5. Общая подготовка (после ЛР № 1)

Шаг 5.1. Проверить инструменты

```
python --version
git --version
```

Зачем это делаем: *`match-case` появился в Python 3.10. Если `python --version` показывает 3.9 или ниже, обновите интерпретатор — иначе конструкция не запустится. Обе версии записываем в отчёт.*

Шаг 5.2. Создать папку проекта

Создайте отдельную папку под этот скрипт (имя — по варианту, см. разделы 6 и 7):

```
mkdir validator-basic
```

Зачем это делаем: *Отдельная папка = отдельный проект. В одну папку складываем код, README и настройки; чужие проекты не смешиваем.*

Шаг 5.3. Создать и активировать виртуальное окружение

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Зачем это делаем: *Окружение изолирует библиотеки проекта. После активации в начале строки терминала появляется (.venv).*

Выберите интерпретатор в VS Code: `Ctrl + Shift + P` → `Python: Select Interpreter` → `.venv`.

6. Вариант А (базовый): «Валидатор технологических параметров скважины»

6.1. Что мы делаем

Создаём консольный скрипт-валидатор. Он запрашивает четыре параметра скважины, каждый вводится ЦИКЛИЧЕСКИ до попадания в допустимый диапазон (границы проверяются через `if-elif-else`), затем классифицирует скважину по обводнённости (тоже `if-elif-else`). Меню программы маршрутизируется через `match-case`.

Это минимальная, но честная задача валидации данных: те же проверки делает любая форма ввода на производстве, только там данные приходят из интерфейса/базы, а у нас — через `input()`.

6.2. Индивидуальные исходные данные

Чтобы работа была индивидуальной, исходные параметры вычисляются из вашего номера `N` (номер по журналу группы; если неизвестен — последние две цифры номера зачётки). Формулы:

```
N   = ваш номер по журналу (или 2 последние цифры зачётки)

Qж  = 40 + 4·N           (дебит жидкости, м³/сут)
W   = 2 + 3·(N % 33)     (обводнённость, %)
Рпл = 16 + 0.5·(N % 30)  (пластовое давление, МПа)
H   = 1200 + 60·(N % 35) (глубина, м)
```

`N % 33`, `N % 30`, `N % 35` — остаток от деления. Подставьте свой `N` и вычислите `Qж`, `W`, `Рпл`, `H` на бумаге — это ваши данные для запуска и ручного прогноза. Запишите их в отчёт. Обратите внимание: обводнённость `W` у разных студентов попадает в РАЗНЫЕ диапазоны классификации (раздел 6.4) — это и есть индивидуальность.

Шаг 1. Создать проект

```
mkdir validator-basic
cd validator-basic
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Создайте в корне папки файл `main.py` (вариант А — одностраничный).

Шаг 2. Написать код

Файл `main.py` — используйте этот каркас как основу:

```
"""Валидатор технологических параметров нефтяной скважины (вариант А)."""

# ---- допустимые границы (константы) ----
Q_RANGE = (0.0, 1000.0)   # дебит жидкости, м³/сут
W_RANGE = (0.0, 100.0)    # обводнённость, %
P_RANGE = (0.0, 100.0)    # пластовое давление, МПа
H_RANGE = (500.0, 4000.0) # глубина, м


def input_bounded(prompt: str, name: str, lo: float, hi: float) -> float:
```

```
    """Циклический ввод числа в диапазоне [lo; hi] с проверкой границ."""
    while True:
        raw = input(prompt).strip().replace(",", ".")
        if raw == "":
            print(f"[пропуск] Пустая строка. Поле '{name}' обязательно.")
            continue
        try:
            value = float(raw)
        except ValueError:
            print(f"[ошибка] '{raw}' — не число. Введите число.")
            continue
        # проверка границ через if-elif-else
        if value < lo:
            print(f"[ошибка] {name} = {value:g} меньше минимума {lo:g}.")
            continue
        elif value > hi:
            print(f"[ошибка] {name} = {value:g} больше максимума {hi:g}.")
            continue
        else:
            return value


def classify_well(w_pct: float) -> str:
    """Классификация скважины по обводнённости (if-elif-else)."""
    if w_pct < 5:
        return "нефтяная (маловодная)"
    elif w_pct < 30:
        return "условно-безводная"
    elif w_pct < 80:
        return "обводнённая"
    else:
        return "водяная (эксплуатация нерациональна)"


def main() -> None:
    print("ВАЛИДАТОР ПАРАМЕТРОВ СКВАЖИНЫ")
    print("=" * 32)

    q = w = p = h = None
    while True:
        print()
        print("МЕНЮ")
        print("1 — ввод параметров")
        print("2 — классификация скважины")
        print("3 — сводка")
        print("0 — выход")
        choice = input("Выбор: ").strip()

        match choice:
            case "1":
                q = input_bounded("Дебит жидкости Qж, м³/сут: ", "Qж", *Q_RANGE)
                w = input_bounded("Обводнённость W, %: ", "W", *W_RANGE)
                p = input_bounded("Пластовое давление Рпл, МПа: ", "Рпл",
*P_RANGE)
```

```
                h = input_bounded("Глубина H, м: ", "H", *H_RANGE)
                print("[ok] Параметры приняты.")
            case "2":
                if w is None:
                    print("[предупреждение] Сначала введите параметры (пункт
1).")
                else:
                    print(f"Обводнённость {w:.1f}% -> {classify_well(w)}")
            case "3":
                if q is None:
                    print("[предупреждение] Сначала введите параметры (пункт
1).")
                else:
                    print(f"Qж={q:.2f} м³/сут | W={w:.2f}% | "
                          f"Рпл={p:.2f} МПа | H={h:.0f} м")
            case "0":
                print("Выход.")
                break
            case _:
                print("[ошибка] Неизвестный пункт меню.")


if __name__ == "__main__":
    main()
```

Зачем это делаем: *`input_bounded` — сердце валидации: цикл `while True` спрашивает, пока значение не пройдёт все проверки, а `if-elif-else` указывает причину отказа. Классификация `classify_well` — та же «лесенка» `if-elif-else`, но уже по диапазонам. Меню маршрутизируется через `match-case`: каждой команде — своя ветка.*

Шаг 3. Запустить и проверить

```
python main.py
```

Введите свои значения `Qж`, `W`, `Рпл`, `H` (из п. 6.2). Пример сеанса (для `N = 5`: Qж = 60, W = 17, Рпл = 18.5, H = 1500):

```
ВАЛИДАТОР ПАРАМЕТРОВ СКВАЖИНЫ
==================================
МЕНЮ
1 — ввод параметров
2 — классификация скважины
3 — сводка
0 — выход
Выбор: 1
Дебит жидкости Qж, м³/сут: 60
Обводнённость W, %: 17
Пластовое давление Рпл, МПа: 18,5
Глубина H, м: 1500
[ok] Параметры приняты.

Выбор: 2
Обводнённость 17.0% -> условно-безводная

Выбор: 3
```

```
Qж=60.00 м³/сут | W=17.00% | Рпл=18.50 МПа | H=1500 м

Выбор: 0
Выход.
```

Теперь намеренно введите НЕДОПУСТИМОЕ значение — например, глубину `-100` или дебит `99999` — и убедитесь, что программа возвращает вас к вводу с понятным сообщением:

```
Глубина H, м: -100
[ошибка] H = -100 меньше минимума 500.
Глубина H, м: 1500
[ok] Параметры приняты.
```

Зачем это делаем: *Провокационный ввод — главный тест валидатора. Программа обязана не упасть, а вежливо попросить значение ещё раз.*

Шаг 4. Ручной прогноз до запуска (обязательно)

ПЕРЕД запуском выпишите на бумаге, какую строку должна вывести программа для пункта «2 — классификация» с вашими данными. Подставьте свою `W` в цепочку:

```
W = 17  ->  5 <= 17 < 30  ->  "условно-безводная"
```

Затем запустите и сверьте. В отчёт добавьте таблицу «прогноз | факт».

Зачем это делаем: *Прогноз до запуска — гарантия того, что вы понимаете цепочку `if-elif-else`, а не просто «нажали кнопку». Именно прогноз спрашивают на защите.*

Шаг 5. Закоммитить в Git

```
git init
git add main.py
git commit -m "Вариант А: валидация, классификация, меню"
```

Добавьте `README.md` (назначение + пример запуска) и сделайте отдельный коммит:

```
git add README.md
git commit -m "Добавить README с примером запуска"
```

Зачем это делаем: *Два коммита с осмысленными сообщениями показывают историю: сначала рабочий код, затем документация.*

7. Вариант Б (продвинутый): «Валидатор технологического режима с маршрутизацией статусов»

7.1. Что мы делаем

Расширяем вариант А до инструмента технолога: добавляем забойное давление, проверяем НЕ ТОЛЬКО границы, но и взаимную непротиворечивость параметров (депрессия на пласт), и маршрутизируем действия через `match-case` по ИТОГОВОМУ СТАТУСУ режима. Логика разнесена по файлам, как в настоящем проекте.

7.2. Индивидуальные исходные данные

К данным варианта А добавляется забойное давление `Рзаб` (тот же номер `N`):

```
Рзаб = 10 + 0.3·(N % 20)   (забойное давление, МПа)
```

Выпишите все пять чисел (`Qж`, `W`, `Рпл`, `Рзаб`, `H`) — это ваш индивидуальный набор. От них зависит и классификация по обводнённости, и статус режима скважины (депрессия `ΔP = Рпл − Рзаб`).

Шаг 1. Создать проект

```
mkdir validator-advanced
cd validator-advanced
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Создайте структуру пакета:

```
validator-advanced/
├── .gitignore
├── README.md
└── src/
    └── well_valid/
        ├── __init__.py
        ├── config.py
        ├── validators.py
        └── main.py
```

Шаг 2. Написать код

Файл `src/well_valid/__init__.py`:

```
"""Валидатор технологического режима нефтяной скважины."""

__version__ = "0.3.0"
```

Файл `src/well_valid/config.py` — допустимые границы отдельно от логики:

```
"""Допустимые границы технологических параметров."""
Q_RANGE = (0.0, 1000.0)    # дебит жидкости, м³/сут
W_RANGE = (0.0, 100.0)     # обводнённость, %
P_RES_RANGE = (0.0, 100.0) # пластовое давление, МПа
P_BHP_RANGE = (0.0, 100.0) # забойное давление, МПа
H_RANGE = (500.0, 4000.0)  # глубина, м
```

Файл `src/well_valid/validators.py` — валидация, классификация и статус режима:

```
"""Валидация: границы (if-elif-else), классификация и статус режима."""
from well_valid import config


def input_bounded(prompt: str, name: str, lo: float, hi: float) -> float:
    """Циклический ввод числа в диапазоне [lo; hi] с проверкой границ."""
    while True:
        raw = input(prompt).strip().replace(",", ".")
        if raw == "":
            print(f"[пропуск] Поле '{name}' обязательно.")
            continue
        try:
            value = float(raw)
        except ValueError:
```

```
            print(f"[ошибка] '{raw}' — не число.")
            continue
        if value < lo:
            print(f"[ошибка] {name} = {value:g} меньше {lo:g}.")
            continue
        elif value > hi:
            print(f"[ошибка] {name} = {value:g} больше {hi:g}.")
            continue
        else:
            return value


def classify_well(w_pct: float) -> str:
    """Классификация по обводнённости (if-elif-else)."""
    if w_pct < 5:
        return "нефтяная (маловодная)"
    elif w_pct < 30:
        return "условно-безводная"
    elif w_pct < 80:
        return "обводнённая"
    else:
        return "водяная (эксплуатация нерациональна)"


def check_mode(p_res: float, p_bhp: float):
    """Определяет статус режима по депрессии (if-elif-else)."""
    drawdown = p_res - p_bhp           # депрессия на пласт, МПа
    if abs(drawdown) <= 0.5:
        return "нулевая", drawdown
    elif drawdown < 0:
        return "фонтанирование", drawdown
    elif drawdown < 2.0:
        return "низкий", drawdown
    else:
        return "норма", drawdown


def print_action(status: str, drawdown: float, q_liq: float) -> None:
    """Маршрутизация действия по статусу через match-case."""
    match status:
        case "норма":
            j = q_liq / drawdown
            print(f"Коэффициент продуктивности J = {j:.3f} м³/(сут·МПа)")
        case "низкий":
            print("Депрессия мала — продуктивность оценена грубо.")
        case "фонтанирование":
            print("(!) Рзаб > Рпл: скважина фонтанирует, расчёт J некорректен.")
        case "нулевая":
            print("Депрессия нулевая — J не определён (деление на ноль!).")
        case _:
            print("Неизвестный статус.")
```

Зачем это делаем: *Обратите внимание на пару инструментов под задачу: `if-elif-else` считает ЧИСЛОВУЮ депрессию и выбирает статус по диапазонам, а `match-case`*

маршрутизирует ДЕЙСТВИЕ по итоговой строке статуса. Границы вынесены в `config.py` — их удобно менять без правки логики.

Файл `src/well_valid/main.py`:

```
"""Инженерный валидатор технологического режима (вариант Б)."""
from well_valid import config
from well_valid import validators


def main() -> None:
    print("ВАЛИДАТОР ТЕХНОЛОГИЧЕСКОГО РЕЖИМА")
    print("=" * 40)

    q = w = p_res = p_bhp = h = None
    while True:
        print()
        print("МЕНЮ")
        print("1 — ввод параметров")
        print("2 — классификация по обводнённости")
        print("3 — режим по давлениям")
        print("4 — сводка")
        print("0 — выход")
        choice = input("Выбор: ").strip()

        match choice:
            case "1":
                q = validators.input_bounded(
                    "Дебит жидкости Qж, м³/сут: ", "Qж", *config.Q_RANGE)
                w = validators.input_bounded(
                    "Обводнённость W, %: ", "W", *config.W_RANGE)
                p_res = validators.input_bounded(
                    "Пластовое давление Рпл, МПа: ", "Рпл", *config.P_RES_RANGE)
                p_bhp = validators.input_bounded(
                    "Забойное давление Рзаб, МПа: ", "Рзаб", *config.P_BHP_RANGE)
                h = validators.input_bounded(
                    "Глубина H, м: ", "H", *config.H_RANGE)
                print("[ok] Параметры приняты.")
            case "2":
                if w is None:
                    print("[предупреждение] Сначала введите параметры (пункт
1).")
                else:
                    print(f"W = {w:.1f}% -> {validators.classify_well(w)}")
            case "3":
                if p_res is None or p_bhp is None:
                    print("[предупреждение] Сначала введите параметры (пункт
1).")
                else:
                    status, dp = validators.check_mode(p_res, p_bhp)
                    print(f"Депрессия ΔP = {dp:.2f} МПа; режим: {status}")
                    validators.print_action(status, dp, q if q is not None else
0.0)
            case "4":
```

```
                if q is None:
                    print("[предупреждение] Сначала введите параметры (пункт
1).")
                else:
                    print(f"Qж={q:.1f} м³/сут | W={w:.1f}% | Рпл={p_res:.2f} МПа
| "
                          f"Рзаб={p_bhp:.2f} МПа | H={h:.0f} м")
            case "0":
                print("Выход.")
                break
            case _:
                print("[ошибка] Неизвестный пункт меню.")


if __name__ == "__main__":
    main()
```

Зачем это делаем: *`match-choice` — маршрутизация меню по командам; `match-status` в `print_action` — маршрутизация действий по статусу режима. Как только появится новый статус, вы добавите новую `case`-ветку — логика не «размажется» по `elif`.*

Шаг 3. Запустить и проверить

```
$env:PYTHONPATH="src"
python -m well_valid.main
```

Пример диалога (для `N = 3`: Qж = 52, W = 11, Рпл = 17.5, Рзаб = 10.9, H = 1380):

```
ВАЛИДАТОР ТЕХНОЛОГИЧЕСКОГО РЕЖИМА
=====================================
Выбор: 1
Дебит жидкости Qж, м³/сут: 52
Обводнённость W, %: 11
Пластовое давление Рпл, МПа: 17,5
Забойное давление Рзаб, МПа: 10,9
Глубина H, м: 1380
[ok] Параметры приняты.

Выбор: 3
Депрессия ΔP = 6.60 МПа; режим: норма
Коэффициент продуктивности J = 7.879 м³/(сут·МПа)
```

Проверьте другой режим: введите `Рзаб` равным `Рпл` (депрессия ноль) или даже больше пластового:

```
Пластовое давление Рпл, МПа: 17,5
Забойное давление Рзаб, МПа: 17,5
Депрессия ΔP = 0.00 МПа; режим: нулевая
Депрессия нулевая — J не определён (деление на ноль!).
```

Шаг 4. Ручной контрольный пересчёт

Покажите ручной расчёт депрессии и статуса для своих данных:

```
ΔP = Рпл − Рзаб = 17,5 − 10,9 = 6,60 МПа  ->  ΔP >= 2.0  ->  "норма"
J = Qж / ΔP = 52 / 6,60 = 7,88 м³/(сут·МПа)
```

Шаг 5. Закоммитить в ветке Git

```
git init
git switch -c feature/validator
git add .
git commit -m "Добавить конфигурацию и валидаторы"
git add src/well_valid/main.py
git commit -m "Добавить меню и маршрутизацию статусов"
git switch -c main
git merge feature/validator
```

8. Обязательные упражнения на понимание (для обоих вариантов)

Семь упражнений выполняют ВСЕ студенты — и в варианте А, и в варианте Б. Результат каждого упражнения обязательно фиксируется в отчёте.

Упражнение 1. Перенастройка границ

В `config.py` (или в константах `main.py`) измените диапазон глубины: сделайте `H_RANGE = (800.0, 3000.0)` вместо `(500.0, 4000.0)`. Запустите и введите глубину, которая раньше была допустима (например, `600`), а теперь не проходит.

Зачем это делаем: *Границы — это данные, а не часть алгоритма. Их переносят в конфиг, чтобы менять безопасно и в одном месте.*

В отчёт: Приведите скриншот, где значение `600` отвергается с сообщением о НОВОЙ границе `800`.

Упражнение 2. Прогноз до запуска

Перед запуском выпишите на бумаге для ВАШЕГО `N`: (а) диапазон классификации, в который попадает ваша `W`; (б) для варианта Б — статус режима по депрессии `ΔP`. Затем запустите и сверьте.

В отчёте приведите таблицу «прогноз | факт» по каждой точке: классификация и статус. Расхождение означает ошибку в вашем понимании цепочки условий.

Упражнение 3. Провокационные вводы

Последовательно введите: пустую строку, текст `abc`, отрицательное число, дробь через запятую `0,25`, очень большое число `99999` и несуществующий пункт меню `9`. Для каждого ввода снимите скриншот реакции программы.

Зачем это делаем: *Намеренно сломанный сценарий доказывает, что валидация и маршрутизация реально работают, а не «написаны для галочки».*

В отчёт: К каждому из шести скриншотов укажите СТРОКУ вашего кода, которая обработала этот ввод (например, `raise` из `except ValueError`, ветку `case _`).

Упражнение 4. Граничные значения обводнённости

Запустите классификацию ровно на границах: `W = 0, 5, 30, 80, 100` (можно по очереди).

Зачем это делаем: *Крайние случаи ловят ошибки «строгой/нестрогой» границы: при `W < 5` значение `5` уже попадает в следующий диапазон.*

В отчёте запишите результаты всех пяти запусков и укажите, какая строка `elif`-цепочки сработала на каждом из них.

Упражнение 5. Расширение меню через match-case

Добавьте в меню новый пункт `5 — справка`, который печатает краткий список команд и их назначение. Реализуйте его НОВОЙ веткой `case "5":`.

Зачем это делаем: *Добавление ветки в `match` должно быть тривиальным — это и есть удобство маршрутизации по значению.*

В отчёт: Приложите код новой ветки и скриншот работы пункта «справка».

Упражнение 6. Новая проверка взаимной непротиворечивости

Добавьте проверку, которой ещё нет в вашей программе:

- вариант А — проверку, что `Рпл` имеет смысл: если `Рпл > 80` МПа, вывести предупреждение «аномально высокое давление, проверьте данные»;
- вариант Б — проверку, что `Рзаб < Рпл + 2,0` (депрессия не слишком мала), через `if-elif-else` или дополнительный статус.

Зачем это делаем: *Проверка связей между полями — следующий уровень валидации: данные корректны по отдельности, но противоречивы вместе.*

В отчёт: Приложите код новой проверки и скриншот, где она срабатывает. Объясните, стоит ли здесь `if-elif-else` или удобнее добавить статус и `match-case`.

Упражнение 7. Вырожденный случай (без падения программы)

Обработайте крайний случай, когда вычисление теряет смысл, — программа должна выдать честный статус, а не `traceback`:

- вариант А — `W = 100` (чистая вода): убедитесь, что классификация не падает;
- вариант Б — `Рзаб = Рпл` (депрессия ноль): статус «нулевая», деление на ноль защищено веткой `case`.

Зачем это делаем: *Устойчивость к вырожденным данным — признак зрелого кода, а не «пример из тетради».*

В отчёт: Приложите скриншот, на котором программа корректно (без traceback) обрабатывает этот случай, и укажите строки, которые защищают от ошибки.

10. Пример оформления отчёта

- Титульная часть: дисциплина, тема, ФИО, группа, дата, вариант, номер по журналу `N`;
- Цель работы;
- Оборудование и ПО с версиями (Python от 3.10, Git, VS Code);
- Индивидуальные исходные данные: ваш `N` и вычисленные `Qж, W, Рпл, Рзаб, H`;
- Ход работы: команды и скриншоты создания проекта, запуска;
- Результаты: вывод программы + таблица «прогноз | факт»;
- Ручной контрольный пересчёт классификации и статуса;
- Ответы на контрольные вопросы (раздел 12);
- Вывод.

Пример начала отчёта (вариант А):

```
----------------------------------------
Отчёт по лабораторной работе № 3

ФИО

Группа
Дисциплина: Программирование и прототипирование ИТ-решений
Вариант А · Номер по журналу N = 5

Исходные данные:
  Qж = 60 м³/сут, W = 17 %, Рпл = 18,5 МПа, H = 1500 м

Прогноз до запуска:
  W = 17 -> 5 <= W < 30 -> «условно-безводная»
Факт: программа вывела «условно-безводная» -> совпало
----------------------------------------
```

11. Частые ошибки и их решение

Симптом	Причина и решение
Программа падает на тексте `abc`	ввод не обёрнут в `try/except`: добавьте конструкцию и сообщение «не число».
`0,25` не принимается, программа падает	не заменена запятая: используйте `.replace(",", ".")` перед `float()`.
Ввод в цикле не выходит / крутится бесконечно	нет `return`/`break` в ветке корректного значения: верните значение сразу после прохождения проверок.
Классификация «съедает» границу (например, `W = 80`)	нестрогая граница оформлена неверно: проверяйте порядок `elif` и знаки `</>=`.
Непонятный пункт меню молча ничего не делает	нет ветки `case _`: добавьте ветку «по умолчанию» с сообщением.
`match` не работает, ошибка синтаксиса	Python ниже 3.10: обновите интерпретатор (`python --version`).
Отступы «съехали» после копирования кода	код скопирован с табуляциями: выровняйте отступы пробелами (4 пробела = 1 уровень).
Коммит не создался, git «nothing to commit»	файлы не добавлены в индекс: сначала `git add .`, затем `git commit`.